DATA STRUCTURES & ALGORITHMS · C++

Hashing
Solved Questions

Every problem from Part 16 of the Hashing guide, in order, with a worked C++ solution, the approach behind it, and the mistake that most often breaks it.

38	3	12	143
PROBLEMS	TIERS	PATTERNS	TESTS PASSING

Prepared for **Hari** · on-campus placement preparation
Companion to **Hashing-Guide.pdf** · code verified against **reference-part16-solutions.cpp**
Every solution in this document was compiled with **g++ -std=c++17 -O2 -Wall** and passes its test cases.
Generated 14 October 2026

Contents

Tier 1 — Learn the mechanics

01 LC 1 · Two Sum
02 LC 217 · Contains Duplicate
03 LC 242 · Valid Anagram
04 LC 383 · Ransom Note
05 LC 387 · First Unique Character in a String
06 LC 205 · Isomorphic Strings
07 LC 290 · Word Pattern
08 LC 349 · Intersection of Two Arrays
09 LC 350 · Intersection of Two Arrays II
10 LC 202 · Happy Number
11 LC 219 · Contains Duplicate II
12 LC 1512 · Number of Good Pairs
13 LC 706 · Design HashMap

Tier 2 — The placement core

14 LC 49 · Group Anagrams
15 LC 128 · Longest Consecutive Sequence
16 LC 560 · Subarray Sum Equals K
17 LC 525 · Contiguous Array
18 LC 974 · Subarray Sums Divisible by K
19 LC 454 · 4Sum II
20 LC 36 · Valid Sudoku
21 LC 380 · Insert Delete GetRandom O(1)
22 LC 146 · LRU Cache
23 LC 438 · Find All Anagrams in a String
24 LC 3 · Longest Substring Without Repeating Characters
25 LC 523 · Continuous Subarray Sum
26 LC 1010 · Pairs of Songs With Total Durations Divisible by 60
27 LC 554 · Brick Wall
28 LC 187 · Repeated DNA Sequences

Tier 3 — Harder, but they do appear

29 LC 41 · First Missing Positive
30 LC 76 · Minimum Window Substring
31 LC 30 · Substring with Concatenation of All Words

32 LC 149 · Max Points on a Line

33 LC 460 · LFU Cache

34 LC 1074 · Number of Submatrices That Sum to Target

35 LC 992 · Subarrays with K Different Integers

36 LC 336 · Palindrome Pairs

37 LC 1044 · Longest Duplicate Substring

38 LC 381 · Insert Delete GetRandom O(1) - Duplicates allowed

Tier 1 — Learn the mechanics

Thirteen problems: existence checks, counting, complements, first and last positions, bijections, and a hash map built by hand. When you can write Two Sum, Valid Anagram and Isomorphic Strings cold, you have the mechanics.

01 LC 1 · Two Sum

EASY Pattern 3 — Complement lookup

PROBLEM

Return the indices of the two numbers that add up to `target`. Exactly one solution exists; an element may not be used twice.

APPROACH

The brute force checks every pair. Turn it around: for each `x`, the partner you need is `target - x` — the **complement**. Keep a map from each value already seen to its index, and ask it for the complement.

Look up **before** inserting the current element, so an element is never paired with itself.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> seen;              // value -> index
        for (int i = 0; i < (int)nums.size(); i++) {
            int need = target - nums[i];           // the complement question
            auto it = seen.find(need);
            if (it != seen.end()) return {it->second, i};
            seen[nums[i]] = i;                     // insert AFTER the lookup
        }
        return {};
    }
};
```

WATCH OUT

Inserting first breaks `[3,2,4]`, target 6: the 3 finds itself and you return `[0,0]`. And sorting + two pointers loses the original indices — that is why this problem is a hash problem. Expect the follow-ups: *the array is sorted* (two pointers, O(1) space) and *return all pairs*.

02 LC 217 · Contains Duplicate

EASY Pattern 1 — Existence check

PROBLEM

Return true if any value appears at least twice.

APPROACH

One pass with a set. `insert` returns a pair whose `.second` says whether the element was **new**, so a single call both checks and records.

Complexity O(n) time · O(n) space (sort + compare neighbours: O(n log n), O(1))

C++ SOLUTION

```cpp
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        unordered_set<int> seen;
        for (int x : nums)
            if (!seen.insert(x).second) return true;   // insert says "already there"
        return false;
    }
};
```

WATCH OUT

`if (s.count(x)) return true; s.insert(x);` is correct but hashes twice. Mention the sorting alternative — interviewers often ask "*what if memory is tight?*"

03 LC 242 · Valid Anagram

EASY Pattern 2 — Frequency counting

PROBLEM

Is `t` an anagram of `s` — the same letters with the same counts?

APPROACH

Only 26 lowercase letters, so an `int[26]` is the hash map. Increment for each character of `s`, decrement for `t`; every count must end at zero. Different lengths fail immediately.

Complexity O(n) time · O(1) space (26 counters)

C++ SOLUTION

```cpp
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) return false;
        int cnt[26] = {0};                      // an array IS a hash map for 26 keys
        for (int i = 0; i < (int)s.size(); i++) {
            cnt[s[i] - 'a']++;
            cnt[t[i] - 'a']--;
        }
        for (int c : cnt) if (c != 0) return false;
        return true;
    }
};
```

WATCH OUT

The follow-up is *Unicode input*: the array no longer fits, so switch to `unordered_map<char32_t, int>` — the algorithm is unchanged. Sorting both strings and comparing is O(n log n) and is the answer interviewers want you to improve on.

04 LC 383 · Ransom Note

EASY Pattern 2 — Supply versus demand

PROBLEM

Can `ransomNote` be built from the letters of `magazine`, each letter used at most once?

APPROACH

Count the magazine's letters (the supply). Walk the note and spend: the moment any count goes negative, demand exceeds supply.

Complexity O(m + n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool canConstruct(string ransomNote, string magazine) {
        int cnt[26] = {0};
        for (char c : magazine) cnt[c - 'a']++;        // supply
        for (char c : ransomNote)
            if (--cnt[c - 'a'] < 0) return false;       // demand exceeds supply
        return true;
    }
};
```

WATCH OUT

Counting both strings and comparing at the end works too, but failing early is cleaner. Counting the **note** as supply is the reversed bug — it answers the opposite question.

05 LC 387 · First Unique Character in a String

EASY Pattern 2 — Two-pass count

PROBLEM

Return the index of the first character that appears exactly once, or −1.

APPROACH

Pass one counts every character. Pass two walks the string **in order** and returns the first index whose count is 1. The second pass must walk the string, not the count table — the table has no notion of position.

Complexity O(n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int firstUniqChar(string s) {
        int cnt[26] = {0};
        for (char c : s) cnt[c - 'a']++;                // pass 1: count
        for (int i = 0; i < (int)s.size(); i++)         // pass 2: first with count 1
            if (cnt[s[i] - 'a'] == 1) return i;
        return -1;
    }
};
```

WATCH OUT

Returning the first letter of the *alphabet* with count 1 is the common slip. For a **stream** (GfG), you cannot make a second pass: keep a queue of candidates and pop from its front while the front's count exceeds 1 — practice TODO 4.

06 LC 205 · Isomorphic Strings

EASY Pattern 6 — Bijection

PROBLEM

Can the characters of `s` be replaced to get `t`, where the replacement is one-to-one (no two characters map to the same character)?

APPROACH

A bijection needs **two** maps: `s → t` and `t → s`. For each position, either both characters are unmapped (record both directions), or both existing mappings must agree with this pair.

Complexity O(n) time · O(1) space (256-entry arrays)

C++ SOLUTION

```cpp
class Solution {
public:
    bool isIsomorphic(string s, string t) {
        // a BIJECTION needs two maps: s->t and t->s. 0 means "not mapped yet".
        char st[256] = {0}, ts[256] = {0};
        for (int i = 0; i < (int)s.size(); i++) {
            unsigned char a = s[i], b = t[i];
            if (st[a] == 0 && ts[b] == 0) { st[a] = b; ts[b] = a; }
            else if (st[a] != (char)b || ts[b] != (char)a) return false;
        }
        return true;
    }
};
```

WATCH OUT

One map only checks that each `s` character is consistent — it misses two different `s` characters mapping to the same `t` character. `"badc"` / `"baba"` returns true with one map, false with two. The first-occurrence signature (`"paper"` → `0,1,0,3,4`) is the alternative.

07 LC 290 · Word Pattern

EASY Pattern 6 — Bijection

PROBLEM

Does the string of words follow the letter pattern, with a one-to-one correspondence between letters and words?

APPROACH

Split the sentence into words with a `stringstream`. If the counts differ, fail. Then the LC 205 check with two maps, `char → string` and `string → char`.

Complexity O(n) time over the total length · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool wordPattern(string pattern, string s) {
        vector<string> words;
        stringstream ss(s);
        for (string w; ss >> w;) words.push_back(w);
        if (words.size() != pattern.size()) return false;
        unordered_map<char, string> p2w;
        unordered_map<string, char> w2p;
        for (int i = 0; i < (int)words.size(); i++) {
            char p = pattern[i];
            const string& w = words[i];
            if (p2w.count(p) && p2w[p] != w) return false;   // p already means something else
            if (w2p.count(w) && w2p[w] != p) return false;   // w already belongs to another p
            p2w[p] = w; w2p[w] = p;
        }
        return true;
    }
};
```

WATCH OUT

Two traps: the **length check** (`"aaa"` against four words), and the **second map** (`"abba"` against `"dog dog dog dog"` passes with only the letter-to-word map). `stringstream >>` also handles multiple spaces, which manual splitting often does not.

08 LC 349 · Intersection of Two Arrays

EASY Pattern 1 — Existence check

PROBLEM

Return the distinct values present in both arrays.

APPROACH

Put the first array into a set. Walk the second: `erase(x)` returns 1 the first time `x` is found and 0 afterwards — so it records the match **and** prevents a duplicate in the output in one call.

Complexity O(m + n) time · O(m) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> intersection(vector<int>& a, vector<int>& b) {
        unordered_set<int> inA(a.begin(), a.end());
        vector<int> res;
        for (int x : b)
            if (inA.erase(x)) res.push_back(x);   // erase returns 1 once, then 0
        return res;
    }
};
```

WATCH OUT

Pushing every `x` that is merely *contained* produces duplicates when the second array repeats a value. The erase trick (or a second set for the output) fixes it.

09 LC 350 · Intersection of Two Arrays II

EASY Pattern 2 — Counts consumed

PROBLEM

Return the intersection **with multiplicity** — each value as many times as it appears in both.

APPROACH

Count the smaller array. Walk the other; each match consumes one count. Counting the smaller array keeps the map small.

Complexity O(m + n) time · O(min(m, n)) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> intersect(vector<int>& a, vector<int>& b) {
        if (a.size() > b.size()) return intersect(b, a);   // count the smaller array
        unordered_map<int, int> cnt;
        for (int x : a) cnt[x]++;
        vector<int> res;
        for (int x : b) {
            auto it = cnt.find(x);
            if (it != cnt.end() && it->second > 0) { res.push_back(x); it->second--; }
        }
        return res;
    }
};
```

WATCH OUT

The follow-ups are the point of the problem: *both arrays sorted* → two pointers, O(1) space; *one array too large for memory* → stream it from disk against the in-memory counts of the smaller one.

10 LC 202 · Happy Number

EASY Pattern 1 — Cycle detection

PROBLEM

Repeatedly replace n by the sum of the squares of its digits. Is it eventually 1?

APPROACH

The sequence either reaches 1 or loops forever. "Loops" means *a value repeats* — keep a set of values seen and stop on a repeat.

Complexity O(log n) per step, few steps · O(number of distinct values) space

C++ SOLUTION

```cpp
class Solution {
    int next(int n) {
        int s = 0;
        for (; n; n /= 10) s += (n % 10) * (n % 10);
        return s;
    }
public:
    bool isHappy(int n) {
        unordered_set<int> seen;                // a repeat means we are in a cycle
        while (n != 1 && !seen.count(n)) {
            seen.insert(n);
            n = next(n);
        }
        return n == 1;
    }
};
```

WATCH OUT

It never diverges: for n ≥ 1000 the next value is much smaller, so the sequence is bounded and must cycle or hit 1. The O(1)-space answer is Floyd's fast/slow pointers from linked lists — name it as the follow-up.

11 LC 219 · Contains Duplicate II

EASY Pattern 4 — Last index

PROBLEM

Are there two equal values at indices at most k apart?

APPROACH

Map each value to its **most recent** index. At each position, if the value was seen and the distance is at most k, done. Then overwrite with the current index — the newest occurrence is the only one that can be close to future elements.

Complexity O(n) time · O(n) space (a sliding set of size k gives O(k))

C++ SOLUTION

```cpp
class Solution {
public:
    bool containsNearbyDuplicate(vector<int>& nums, int k) {
        unordered_map<int, int> last;            // value -> most recent index
        for (int i = 0; i < (int)nums.size(); i++) {
            auto it = last.find(nums[i]);
            if (it != last.end() && i - it->second <= k) return true;
            last[nums[i]] = i;                   // keep only the LATEST position
        }
        return false;
    }
};
```

WATCH OUT

Keeping the **first** index instead of the last misses `[1,0,1,1]`, k = 1. The window-set version — insert, and erase `nums[i − k]` once the window is full — is the O(k)-space alternative worth mentioning.

12 LC 1512 · Number of Good Pairs

EASY Pattern 2 — Pairs in one pass

PROBLEM

Count pairs i < j with `nums[i] == nums[j]`.

APPROACH

Each new element forms a pair with every earlier copy of itself. So add the current count **before** incrementing it. The final total equals the sum of c·(c−1)/2 over all counts.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int numIdenticalPairs(vector<int>& nums) {
        unordered_map<int, int> cnt;
        int pairs = 0;
        for (int x : nums) {
            pairs += cnt[x];        // x pairs with every earlier copy of itself
            cnt[x]++;
        }
        return pairs;
    }
};
```

WATCH OUT

Incrementing first counts each element as a pair with itself. This one-pass shape is the template for *pairs with sum k* (`cnt[k - x]`) and *pairs divisible by 60* (LC 1010) — learn it here.

13 LC 706 · Design HashMap

EASY Build the table by hand

PROBLEM

Implement `put`, `get` and `remove` for integer keys without any built-in hash table.

APPROACH

Separate chaining: a prime number of buckets, each a list of `(key, value)` pairs. `put` overwrites an existing key in its bucket or appends; `get` scans the bucket; `remove` unlinks.

A prime bucket count spreads keys that share common factors.

Complexity O(1) average per operation (n / B keys per bucket) · O(B + n) space

C++ SOLUTION

```cpp
class MyHashMap {
    static const int B = 10007;                      // prime number of buckets
    vector<list<pair<int, int>>> buckets;            // separate chaining
    list<pair<int, int>>& bucket(int key) { return buckets[key % B]; }
public:
    MyHashMap() : buckets(B) {}
    void put(int key, int value) {
        for (auto& kv : bucket(key))
            if (kv.first == key) { kv.second = value; return; }   // overwrite
        bucket(key).push_back({key, value});
    }
    int get(int key) {
        for (auto& kv : bucket(key)) if (kv.first == key) return kv.second;
        return -1;
    }
    void remove(int key) {
        bucket(key).remove_if([key](const pair<int, int>& kv) { return kv.first == key; });
    }
};
```

WATCH OUT

`put` must **overwrite**, not append a duplicate — otherwise `get` returns the stale value. Expect: *what is your load factor?*, *how would you resize?*, *how would open addressing handle `remove`?* (tombstones — guide Part 1.3).

Tier 2 — The placement core

These are what actually get asked. Group Anagrams, Longest Consecutive Sequence, Subarray Sum Equals K, Longest Substring Without Repeating Characters and LRU Cache appear in nearly every on-campus drive.

14 LC 49 · Group Anagrams

MEDIUM Pattern 5 — Signature

PROBLEM

Group the strings that are anagrams of each other.

APPROACH

Anagrams share a **signature**. The sorted string works (O(L log L) per word); the 26 letter counts written into a 26-character string work in O(L). Map signature → list of words.

Complexity O(n · L) time with the count key · O(n · L) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> groups;
        for (const string& s : strs) {
            // signature: 26 counts, written out as a string key -> O(L) per word
            string key(26, 0);
            for (char c : s) key[c - 'a']++;
            groups[key].push_back(s);
        }
        vector<vector<string>> res;
        for (auto& [key, g] : groups) res.push_back(move(g));
        return res;
    }
};
```

WATCH OUT

Building the key as `to_string(c0) + to_string(c1) + …` without separators is ambiguous — counts 1,11 and 11,1 both give `"111"`. A fixed-width key (one `char` per letter) or a separator avoids it. Groups come back in no particular order.

15 LC 128 · Longest Consecutive Sequence

MEDIUM Pattern 9 — Run starts

PROBLEM

Return the length of the longest run of consecutive integers, in O(n).

APPROACH

Put everything in a set. A value x **starts** a run only if `x − 1` is absent. Only from starts, walk `x + 1, x + 2, …` while present.

Each value is walked over once — from the start of its own run — so the nested loop is O(n) in total.

Complexity O(n) average time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());
        int best = 0;
        for (int x : s) {
            if (s.count(x - 1)) continue;        // not the START of a run: skip
            int len = 1;
            while (s.count(x + len)) len++;      // walk the run once, from its start
            best = max(best, len);
        }
        return best;
    }
};
```

WATCH OUT

Iterate over the **set**, not the input: with many duplicates of a run's start, iterating the input repeats the walk and degrades towards O(n²). Sorting is O(n log n) and is what the 'in O(n)' requirement rules out.

16 LC 560 · Subarray Sum Equals K

MEDIUM Pattern 7 — Prefix sum + hash map

PROBLEM

Count the contiguous subarrays whose sum is exactly k. Values may be negative.

APPROACH

A subarray sum is a difference of two prefix sums. At each position, count the earlier prefixes equal to `prefix − k` — a map from prefix sum to how many times it has occurred.

Seed the map with `{0: 1}`: the empty prefix, so subarrays starting at index 0 are counted.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<int, int> seen{{0, 1}};    // prefix sum -> how many times; empty prefix
        int prefix = 0, count = 0;
        for (int x : nums) {
            prefix += x;
            // a subarray (j, i] sums to k  <=>  prefix[j] == prefix[i] - k
            auto it = seen.find(prefix - k);
            if (it != seen.end()) count += it->second;
            seen[prefix]++;
        }
        return count;
    }
};
```

WATCH OUT

A sliding window is **wrong** here — with negative numbers, shrinking the window does not reliably lower the sum. Forgetting the `{0: 1}` seed misses every subarray that starts at index 0: `[1,1,1]`, k = 2 must be 2.

17 LC 525 · Contiguous Array

MEDIUM Pattern 7 — ±1 balance, first index

PROBLEM

Return the length of the longest subarray with equally many 0s and 1s.

APPROACH

Replace 0 by −1: equal counts means **sum zero**, so two equal prefix sums enclose a valid subarray. For the **longest**, remember the **first** index of each balance and never overwrite it. Seed `{0: −1}`.

Complexity O(n) time · O(n) space (an array of size 2n + 1 works too)

C++ SOLUTION

```cpp
class Solution {
public:
    int findMaxLength(vector<int>& nums) {
        unordered_map<int, int> first{{0, -1}};  // balance -> FIRST index it was seen
        int bal = 0, best = 0;
        for (int i = 0; i < (int)nums.size(); i++) {
            bal += nums[i] ? 1 : -1;             // 1 -> +1, 0 -> -1
            auto it = first.find(bal);
            if (it != first.end()) best = max(best, i - it->second);
            else first[bal] = i;                 // never overwrite: earliest = longest
        }
        return best;
    }
};
```

WATCH OUT

Overwriting the stored index on every visit makes every distance 0 — store only when absent. The seed is `−1`, not `0`, because the empty prefix ends *before* index 0.

18 LC 974 · Subarray Sums Divisible by K

MEDIUM Pattern 7 — Remainders

PROBLEM

Count the subarrays whose sum is divisible by k.

APPROACH

Two prefixes with **equal remainders** mod k enclose a sum divisible by k. Count remainders in an array of size k (seed `cnt[0] = 1`); each new prefix pairs with every earlier prefix of the same remainder.

Complexity O(n) time · O(k) space

C++ SOLUTION

```cpp
class Solution {
public:
    int subarraysDivByK(vector<int>& nums, int k) {
        vector<int> cnt(k, 0);                  // remainder -> count (array as map)
        cnt[0] = 1;
        int prefix = 0, res = 0;
        for (int x : nums) {
            prefix = ((prefix + x) % k + k) % k; // keep the remainder NON-negative
            res += cnt[prefix];                  // equal remainders -> divisible gap
            cnt[prefix]++;
        }
        return res;
    }
};
```

WATCH OUT

In C++ `-1 % 5 == -1`. A negative prefix gives a negative remainder that never meets its positive twin — normalise with `((p % k) + k) % k`. The test `[-1,2,9]`, k = 2 exists to catch exactly this.

19 LC 454 · 4Sum II

MEDIUM Pattern 3 — Meet in the middle

PROBLEM

Given four arrays of length n, count tuples (i, j, k, l) with `a[i] + b[j] + c[k] + d[l] == 0`.

APPROACH

Split the four arrays into two pairs. Count every `a + b` sum in a map (n^2 entries). Then for every `c + d`, add the count of `−(c + d)`.

Complexity $O(n^2)$ time · $O(n^2)$ space (down from $O(n^4)$)

C++ SOLUTION

```cpp
class Solution {
public:
    int fourSumCount(vector<int>& a, vector<int>& b, vector<int>& c, vector<int>& d) {
        unordered_map<int, int> ab;              // every a+b, with multiplicity
        for (int x : a) for (int y : b) ab[x + y]++;
        int res = 0;
        for (int x : c)
            for (int y : d) {
                auto it = ab.find(-(x + y));     // meet in the middle
                if (it != ab.end()) res += it->second;
            }
        return res;
    }
};
```

WATCH OUT

Store **counts**, not a set — equal sums from different index pairs are different tuples. *Meet in the middle* is the general idea; it also turns $O(2^n)$ subset searches into O(2^(n/2)).

20 LC 36 · Valid Sudoku

MEDIUM Pattern 1 — Three seen-tables

PROBLEM

Check that no filled row, column or 3 × 3 box of a partially filled Sudoku contains a repeated digit.

APPROACH

Three tables of *seen* flags — per row, per column, per box — each indexed by digit. For each filled cell, if any of its three tables already has the digit, fail; otherwise mark all three. The box index is `(r / 3) * 3 + c / 3`.

Complexity O(81) = O(1) time and space

C++ SOLUTION

```cpp
class Solution {
public:
    bool isValidSudoku(vector<vector<char>>& board) {
        bool row[9][9] = {}, col[9][9] = {}, box[9][9] = {};
        for (int r = 0; r < 9; r++)
            for (int c = 0; c < 9; c++) {
                if (board[r][c] == '.') continue;
                int d = board[r][c] - '1', b = (r / 3) * 3 + c / 3;
                if (row[r][d] || col[c][d] || box[b][d]) return false;   // seen before
                row[r][d] = col[c][d] = box[b][d] = true;
            }
        return true;
    }
};
```

WATCH OUT

This only checks the filled cells — it does **not** check that the puzzle is solvable, and the problem does not ask it to. Getting the box formula wrong (`r / 3 + c / 3`) merges different boxes.

21 LC 380 · Insert Delete GetRandom O(1)

MEDIUM Pattern 10 — Map + vector

PROBLEM

Design a set supporting `insert`, `remove` and `getRandom` (uniform) in average O(1).

APPROACH

`getRandom` needs indexing → a **vector**. `remove` needs to find a value's index → a **map from value to index**. To delete from the middle in O(1), move the **last** element into the hole, update its index in the map, and pop the back.

> **Complexity** O(1) average per operation · O(n) space

C++ SOLUTION

```cpp
class RandomizedSet {
    vector<int> vals;                       // dense: random access by index
    unordered_map<int, int> pos;            // value -> its index in vals
public:
    bool insert(int val) {
        if (pos.count(val)) return false;
        pos[val] = vals.size();
        vals.push_back(val);
        return true;
    }
    bool remove(int val) {
        auto it = pos.find(val);
        if (it == pos.end()) return false;
        int i = it->second, last = vals.back();
        vals[i] = last;  pos[last] = i;     // move the last element into the hole
        vals.pop_back(); pos.erase(val);    // erase val LAST (it may equal `last`)
        return true;
    }
    int getRandom() { return vals[rand() % vals.size()]; }
};
```

WATCH OUT

Erase `val` from the map **after** updating `pos[last]`: when `val` *is* the last element, updating first and erasing second is the only order that leaves the map correct. A `set` cannot do `getRandom` in O(1) — no random access.

22 LC 146 · LRU Cache

MEDIUM Pattern 10 — Map + doubly linked list

PROBLEM

Design a cache of fixed capacity with O(1) `get` and `put` that evicts the least recently used key when full.

APPROACH

A doubly linked list keeps keys in recency order (front = most recent). A map from key to **list iterator** finds any node in O(1). `get` and `put` move the node to the front with `splice`; eviction takes the back.

Complexity O(1) per operation · O(capacity) space

C++ SOLUTION

```cpp
class LRUCache {
    int cap;
    list<pair<int, int>> order;                              // front = most recent
    unordered_map<int, list<pair<int, int>>::iterator> where;  // key -> its list node
public:
    LRUCache(int capacity) : cap(capacity) {}
    int get(int key) {
        auto it = where.find(key);
        if (it == where.end()) return -1;
        order.splice(order.begin(), order, it->second);  // move to front, O(1)
        return it->second->second;
    }
    void put(int key, int value) {
        auto it = where.find(key);
        if (it != where.end()) {
            it->second->second = value;
            order.splice(order.begin(), order, it->second);
            return;
        }
        if ((int)order.size() == cap) {                  // evict least recent
            where.erase(order.back().first);
            order.pop_back();
        }
        order.push_front({key, value});
        where[key] = order.begin();
    }
};
```

WATCH OUT

On eviction, read the back node's key and erase it from the **map before** popping the list. `std::list` iterators stay valid through `splice`, which is why it is the right container. Be ready to write it with a hand-made node struct and sentinel head/tail if `std::list` is not allowed.

23 LC 438 · Find All Anagrams in a String

MEDIUM Pattern 8 — Fixed window

PROBLEM

Return every start index in `s` of a substring that is an anagram of `p`.

APPROACH

A window of exactly `|p|` characters slides along `s`, with 26 counts for the window and 26 for `p`. Each step adds the entering character and removes the leaving one; equal tables mean an anagram starts at `i − m + 1`.

Complexity O(26 · n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        vector<int> res;
        int n = s.size(), m = p.size();
        if (m > n) return res;
        int need[26] = {0}, have[26] = {0};
        for (char c : p) need[c - 'a']++;
        for (int i = 0; i < n; i++) {
            have[s[i] - 'a']++;                          // char enters the window
            if (i >= m) have[s[i - m] - 'a']--;          // char leaves the window
            if (i >= m - 1 && equal(need, need + 26, have)) res.push_back(i - m + 1);
        }
        return res;
    }
};
```

WATCH OUT

Rebuilding the window's counts at every position is O(n · m). Keep a running `matches` count of letters whose two counts agree to make each step O(1) — a natural follow-up. Handle `|p| > |s|` up front.

24 LC 3 · Longest Substring Without Repeating Characters

MEDIUM Pattern 8 — Last index inside the window

PROBLEM

Return the length of the longest substring with all distinct characters.

APPROACH

Remember each character's **last index**. When the incoming character was last seen **inside** the window (index ≥ `left`), move `left` just past that occurrence. The window is always repeat-free, and its length is a candidate.

Complexity O(n) time · O(1) space (256 entries)

C++ SOLUTION

```cpp
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        vector<int> last(256, -1);              // char -> last index seen
        int best = 0, left = 0;
        for (int i = 0; i < (int)s.size(); i++) {
            unsigned char c = s[i];
            if (last[c] >= left) left = last[c] + 1;   // repeat INSIDE the window: jump
            last[c] = i;
            best = max(best, i - left + 1);
        }
        return best;
    }
};
```

WATCH OUT

Without the `>= left` check, a stale occurrence *outside* the window drags `left` backwards: `"abba"` answers 3 instead of 2. The set-based version (erase from the left until the repeat is gone) is also O(n); the jump version is shorter.

25 LC 523 · Continuous Subarray Sum

MEDIUM Pattern 7 — Remainder, first index

PROBLEM

Is there a subarray of length at least 2 whose sum is a multiple of k?

APPROACH

Equal prefix remainders mod k enclose a multiple of k. Store the **first** index of each remainder (seed `{0: −1}`) and accept when the current index is at least 2 past it.

Complexity O(n) time · O(min(n, k)) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool checkSubarraySum(vector<int>& nums, int k) {
        unordered_map<int, int> first{{0, -1}};   // remainder -> earliest index
        int prefix = 0;
        for (int i = 0; i < (int)nums.size(); i++) {
            prefix = (prefix + nums[i]) % k;
            auto it = first.find(prefix);
            if (it != first.end()) {
                if (i - it->second >= 2) return true;   // length at least 2
            } else {
                first[prefix] = i;                      // keep the EARLIEST
            }
        }
        return false;
    }
};
```

WATCH OUT

Keep the **earliest** index — overwriting it turns `[5,0,0,0]`, k = 3 into a false negative, because the long enough window starts from the first occurrence. The length check is `i − first ≥ 2`, not `> 1` on the wrong side.

26 LC 1010 · Pairs of Songs With Total Durations Divisible by 60

MEDIUM Pattern 3 — Complement remainder

PROBLEM

Count pairs i < j whose durations sum to a multiple of 60.

APPROACH

Two Sum on remainders. A song with remainder r pairs with every earlier song of remainder `60 − r` — and remainder 0 pairs with remainder 0. Count remainders in an array of 60.

Complexity O(n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int numPairsDivisibleBy60(vector<int>& time) {
        int cnt[60] = {0};
        int pairs = 0;
        for (int t : time) {
            int r = t % 60;
            pairs += cnt[(60 - r) % 60];    // the complement remainder; (60-0)%60 = 0
            cnt[r]++;
        }
        return pairs;
    }
};
```

WATCH OUT

`60 − r` is 60 when r = 0, which is out of range: use `(60 − r) % 60`. Counting all remainders first and multiplying needs the special cases r = 0 and r = 30 (n·(n−1)/2); the one-pass version handles them for free.

27 LC 554 · Brick Wall

MEDIUM Pattern 2 — Count edge positions

PROBLEM

Draw a vertical line through a brick wall crossing the fewest bricks. Return that number.

APPROACH

A line crosses a row's brick unless it passes through that row's **edge**. Count, over all rows, how many rows have an edge at each x position. The best line goes where most edges line up: answer = rows − max count.

Complexity O(total bricks) time · O(distinct edge positions) space

C++ SOLUTION

```cpp
class Solution {
public:
    int leastBricks(vector<vector<int>>& wall) {
        unordered_map<int, int> edges;      // x position -> rows with an edge there
        int most = 0;
        for (auto& row : wall) {
            int x = 0;
            for (int i = 0; i + 1 < (int)row.size(); i++) {   // skip the wall's right edge
                x += row[i];
                most = max(most, ++edges[x]);
            }
        }
        return (int)wall.size() - most;     // cut where the most edges line up
    }
};
```

WATCH OUT

Skip each row's **last** brick — its right edge is the wall's edge, where lines are not allowed. Positions can be up to $2^{31} - 1$, so a map, not an array.

28 LC 187 · Repeated DNA Sequences

MEDIUM Pattern 11 — Perfect rolling encoding

PROBLEM

Return every 10-letter substring that occurs more than once in a DNA string.

APPROACH

Four letters need two bits each, so a 10-letter window fits exactly in 20 bits. Roll: shift in two bits, mask to 20. That integer **is** the window — a perfect hash. Count each code and report it on its **second** sighting.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<string> findRepeatedDnaSequences(string s) {
        // 4 letters -> 2 bits each; a 10-letter window fits in 20 bits exactly,
        // so this "hash" is PERFECT: no collisions possible
        auto code = [](char c) { return c == 'A' ? 0 : c == 'C' ? 1 : c == 'G' ? 2 : 3; };
        unordered_map<int, int> seen;
        vector<string> res;
        int h = 0, mask = (1 << 20) - 1;
        for (int i = 0; i < (int)s.size(); i++) {
            h = ((h << 2) | code(s[i])) & mask;     // roll: drop the oldest letter
            if (i >= 9 && ++seen[h] == 2)           // report on the SECOND sighting only
                res.push_back(s.substr(i - 9, 10));
        }
        return res;
    }
};
```

WATCH OUT

Report when the count becomes exactly 2, not ≥ 2, or a sequence appearing three times is output twice. Hashing `substr(i, 10)` into a set of strings also works but is ten times the work and memory.

Tier 3 — Harder, but they do appear

The same ideas with more moving parts: the array as its own table, harder windows, exact keys, LFU, prefix sums in 2-D, and rolling hashes with verification.

29 LC 41 · First Missing Positive

HARD Pattern 12 — Cyclic placement

PROBLEM

Return the smallest missing positive integer, in O(n) time and **O(1) extra space**.

APPROACH

The answer is in 1 … n + 1, so the array can be its own hash table: value v belongs at index v − 1. Swap each in-range value to its home slot until the current slot is settled. Then the first index i not holding i + 1 gives the answer.

Complexity O(n) time — each swap places one value permanently · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int firstMissingPositive(vector<int>& a) {
        int n = a.size();
        // the array is its own hash table: value v belongs at index v-1
        for (int i = 0; i < n; i++)
            while (a[i] >= 1 && a[i] <= n && a[a[i] - 1] != a[i])
                swap(a[i], a[a[i] - 1]);            // send a[i] home
        for (int i = 0; i < n; i++)
            if (a[i] != i + 1) return i + 1;        // first slot with the wrong owner
        return n + 1;
    }
};
```

WATCH OUT

The loop condition must be `a[a[i] − 1] != a[i]`, not `a[i] != i + 1`: with a duplicate, the second condition swaps two equal values forever. A `set` solution is easy but is O(n) space, which the problem forbids.

30 LC 76 · Minimum Window Substring

HARD Pattern 8 — Need / missing counter

PROBLEM

Return the shortest substring of `s` containing every character of `t`, with multiplicity.

APPROACH

`need[c]` holds how many more of c the window requires; `missing` counts the total still required. Expand right; a character whose need was positive reduces `missing`. Once `missing == 0`, shrink left as far as validity allows, recording the best window.

Complexity O(|s| + |t|) time · O(1) space (128 counters)

C++ SOLUTION

```cpp
class Solution {
public:
    string minWindow(string s, string t) {
        int need[128] = {0};
        for (char c : t) need[(int)c]++;
        int missing = t.size();             // characters of t still not covered
        int bestL = 0, bestLen = INT_MAX, left = 0;
        for (int right = 0; right < (int)s.size(); right++) {
            if (need[(int)s[right]]-- > 0) missing--;          // a useful character
            while (missing == 0) {                             // valid: try to shrink
                if (right - left + 1 < bestLen) { bestLen = right - left + 1; bestL = left; }
                if (++need[(int)s[left]] > 0) missing++;       // dropped a needed char
                left++;
            }
        }
        return bestLen == INT_MAX ? "" : s.substr(bestL, bestLen);
    }
};
```

WATCH OUT

`need` going negative is fine — it means surplus copies, and it is exactly what lets the shrink step know when a removal hurts (`++need > 0`). Comparing whole count tables at every step is O(52 · n) and harder to get right.

31 LC 30 · Substring with Concatenation of All Words

HARD Pattern 8 — Word-sized windows

PROBLEM

All words have the same length. Return every index where a substring is a concatenation of all the words, each exactly once, in any order.

APPROACH

Run w separate sliding windows, one per offset 0 … w − 1, each stepping a whole word at a time. Count words in the window; an unknown word resets it; too many copies of a word shrinks from the left. When the window holds k words, record its start.

Complexity O(n · w) time · O(k · w) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> findSubstring(string s, vector<string>& words) {
        vector<int> res;
        int w = words[0].size(), k = words.size(), n = s.size();
        unordered_map<string, int> need;
        for (auto& x : words) need[x]++;
        for (int off = 0; off < w; off++) {           // w separate word-aligned grids
            unordered_map<string, int> have;
            int left = off, used = 0;
            for (int right = off; right + w <= n; right += w) {
                string word = s.substr(right, w);
                if (!need.count(word)) { have.clear(); used = 0; left = right + w; continue; }
                have[word]++; used++;
                while (have[word] > need[word]) {     // too many copies: shrink
                    have[s.substr(left, w)]--; used--; left += w;
                }
                if (used == k) res.push_back(left);
            }
        }
        return res;
    }
};
```

WATCH OUT

Restarting a fresh check at every index is O(n · k · w) and times out. The words may repeat (`["word","good","best","word"]`), so compare **counts**, not membership.

32 LC 149 · Max Points on a Line

HARD Pattern 5 — Reduced slope as the key

PROBLEM

Return the maximum number of points lying on one straight line.

APPROACH

Fix each point as an anchor and count the others by **slope**. Represent the slope exactly as the pair `(dy, dx)` divided by their gcd, with one sign convention, and use it as a map key. The best line through the anchor is the largest count + 1.

Complexity O(n² log V) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int maxPoints(vector<vector<int>>& p) {
        int n = p.size(), best = min(n, 1);
        for (int i = 0; i < n; i++) {
            map<pair<int, int>, int> slopes;          // reduced (dy, dx) -> count
            for (int j = i + 1; j < n; j++) {
                int dy = p[j][1] - p[i][1], dx = p[j][0] - p[i][0];
                int g = gcd(dy, dx);                  // exact slope, no doubles
                dy /= g; dx /= g;
                if (dx < 0 || (dx == 0 && dy < 0)) { dx = -dx; dy = -dy; }   // one sign form
                best = max(best, ++slopes[{dy, dx}] + 1);   // +1 for point i itself
            }
        }
        return best;
    }
};
```

WATCH OUT

A `double` slope fails on precision and on vertical lines. Normalise the sign (`dx > 0`, or `dx == 0, dy > 0`), or `(1, 2)` and `(−1, −2)` become different keys. `std::pair` has no `std::hash`, so `map` or an encoded key.

33 LC 460 · LFU Cache

HARD Pattern 10 — A list per frequency

PROBLEM

A cache that evicts the least **frequently** used key (ties: least recently used), all in O(1).

APPROACH

`data` maps key → (value, freq, list iterator); `byFreq` maps freq → keys, most recent first; `minFreq` is the lowest frequency present. Evict from the back of `byFreq[minFreq]`.

Complexity O(1) per operation · O(capacity) space

C++ SOLUTION

```cpp
class LFUCache {
    struct Entry { int value, freq; list<int>::iterator it; };
    int cap, minFreq = 0;
    unordered_map<int, Entry> data;              // key -> value, freq, place in its list
    unordered_map<int, list<int>> byFreq;        // freq -> keys, most recent at front
    void touch(int key) {                        // bump a key's frequency by one
        Entry& e = data[key];
        byFreq[e.freq].erase(e.it);
        if (byFreq[e.freq].empty()) {
            byFreq.erase(e.freq);
            if (minFreq == e.freq) minFreq++;
        }
        byFreq[++e.freq].push_front(key);
        e.it = byFreq[e.freq].begin();
    }
public:
    LFUCache(int capacity) : cap(capacity) {}
    int get(int key) {
        if (!data.count(key)) return -1;
        touch(key); return data[key].value;
    }
    void put(int key, int value) {
        if (cap == 0) return;
        if (data.count(key)) { data[key].value = value; touch(key); return; }
        if ((int)data.size() == cap) {           // evict: lowest freq, then least recent
            int victim = byFreq[minFreq].back();
            byFreq[minFreq].pop_back();
            if (byFreq[minFreq].empty()) byFreq.erase(minFreq);
            data.erase(victim);
        }
        byFreq[1].push_front(key);
        data[key] = {value, 1, byFreq[1].begin()};
        minFreq = 1;                             // a new key always has the lowest freq
    }
};
```

WATCH OUT

Bump `minFreq` only if the list just emptied **was** the minimum. Capacity 0 rejects every `put`.

34 LC 1074 · Number of Submatrices That Sum to Target

HARD Pattern 7 — LC 560 in two dimensions

PROBLEM

Count the non-empty submatrices whose sum equals the target.

APPROACH

Fix a top row and a bottom row. Collapse the rows between them into one array of column sums (updated incrementally as `bottom` grows). Counting submatrices between these rows is now LC 560 on that array.

Complexity O(R² · C) time · O(C) space (put the smaller dimension in R²)

C++ SOLUTION

```cpp
class Solution {
public:
    int numSubmatrixSumTarget(vector<vector<int>>& m, int target) {
        int R = m.size(), C = m[0].size(), res = 0;
        for (int top = 0; top < R; top++) {
            vector<int> colSum(C, 0);                // column sums of rows top..bottom
            for (int bottom = top; bottom < R; bottom++) {
                for (int c = 0; c < C; c++) colSum[c] += m[bottom][c];
                // now it is LC 560 on a 1-D array
                unordered_map<int, int> seen{{0, 1}};
                int prefix = 0;
                for (int c = 0; c < C; c++) {
                    prefix += colSum[c];
                    auto it = seen.find(prefix - target);
                    if (it != seen.end()) res += it->second;
                    seen[prefix]++;
                }
            }
        }
        return res;
    }
};
```

WATCH OUT

Recomputing column sums from scratch for every row pair adds a factor of R. If R > C, transpose first — the cost is quadratic in the dimension you fix. Reset the prefix map for every row pair.

35 LC 992 · Subarrays with K Different Integers

HARD Pattern 8 — exactly = atMost − atMost

PROBLEM

Count subarrays containing exactly k distinct values.

APPROACH

"Exactly k" does not shrink nicely, but "at most k" does: for each right edge, the shortest valid left edge gives `right − left + 1` subarrays ending there. Then `exactly(k) = atMost(k) − atMost(k − 1)`.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
    // number of subarrays with AT MOST k distinct values
    int atMost(const vector<int>& a, int k) {
        unordered_map<int, int> cnt;
        int left = 0, res = 0;
        for (int right = 0; right < (int)a.size(); right++) {
            if (cnt[a[right]]++ == 0) k--;            // a new distinct value
            while (k < 0) {
                if (--cnt[a[left]] == 0) k++;         // a value fully left the window
                left++;
            }
            res += right - left + 1;                  // every subarray ending at right
        }
        return res;
    }
public:
    int subarraysWithKDistinct(vector<int>& nums, int k) {
        return atMost(nums, k) - atMost(nums, k - 1);   // exactly = atMost(k) - atMost(k-1)
    }
};
```

WATCH OUT

Track the distinct count when a count goes 0 → 1 or 1 → 0, or erase zero-count keys if you use `map.size()`. The identity `exactly = atMost(k) − atMost(k − 1)` works for any count that grows monotonically with the window — learn it as a tool.

36 LC 336 · Palindrome Pairs

HARD Pattern 5 — Reversed words as keys

PROBLEM

Given distinct words, return every pair (i, j) such that `words[i] + words[j]` is a palindrome.

APPROACH

Map each **reversed** word to its index. Split each word at every position into left + right. If left is a palindrome, a word equal to reverse(right) placed **before** it works. If right is a palindrome, a word equal to reverse(left) placed **after** it works.

Complexity O(n · L²) time · O(n · L) space

C++ SOLUTION

```cpp
class Solution {
    bool isPal(const string& s, int l, int r) {
        while (l < r) if (s[l++] != s[r--]) return false;
        return true;
    }
public:
    vector<vector<int>> palindromePairs(vector<string>& words) {
        unordered_map<string, int> idx;               // reversed word -> its index
        for (int i = 0; i < (int)words.size(); i++)
            idx[string(words[i].rbegin(), words[i].rend())] = i;
        vector<vector<int>> res;
        for (int i = 0; i < (int)words.size(); i++) {
            const string& w = words[i];
            int L = w.size();
            for (int j = 0; j <= L; j++) {            // split w = w[0..j) + w[j..L)
                // left part is a palindrome: reverse(right) + w is a palindrome
                if (isPal(w, 0, j - 1)) {
                    auto it = idx.find(w.substr(j));
                    if (it != idx.end() && it->second != i) res.push_back({it->second, i});
                }
                // right part is a palindrome: w + reverse(left) is a palindrome
                if (j < L && isPal(w, j, L - 1)) {    // j < L avoids double counting
                    auto it = idx.find(w.substr(0, j));
                    if (it != idx.end() && it->second != i) res.push_back({i, it->second});
                }
            }
        }
        return res;
    }
};
```

WATCH OUT

Skip the second check when the split is at the very end (`j == L`), or pairs found by both checks are reported twice. The empty string pairs with every palindrome in both orders. Checking all n² pairs is O(n² · L) and times out.

37 LC 1044 · Longest Duplicate Substring

HARD Pattern 11 — Binary search + rolling hash

PROBLEM

Return a longest substring that occurs at least twice (occurrences may overlap).

APPROACH

If length L repeats, so does L − 1, so **binary search on L**. To test a length, roll a polynomial hash over every window, and on a hash match **compare the actual text** before accepting.

Complexity O(n log n) expected time · O(n) space

C++ SOLUTION

```cpp
class Solution {
    string s;
    // start of some substring of length L that appears twice, or -1
    int dupOfLength(int L) {
        typedef unsigned long long u64;              // arithmetic mod 2^64, for free
        const u64 B = 131;
        u64 h = 0, pw = 1;
        for (int i = 0; i < L; i++) { h = h * B + s[i]; pw *= B; }
        unordered_map<u64, vector<int>> seen{{h, {0}}};
        for (int i = L; i < (int)s.size(); i++) {
            h = h * B + s[i] - pw * s[i - L];        // roll: add right, drop left
            int st = i - L + 1;
            auto it = seen.find(h);
            if (it != seen.end())
                for (int j : it->second)             // equal hash: VERIFY the text
                    if (s.compare(j, L, s, st, L) == 0) return st;
            seen[h].push_back(st);
        }
        return -1;
    }
public:
    string longestDupSubstring(string str) {
        s = str;
        int lo = 1, hi = (int)s.size() - 1, bestStart = 0, bestLen = 0;
        while (lo <= hi) {                           // if length L repeats, so does L-1
            int mid = lo + (hi - lo) / 2;
            int st = dupOfLength(mid);
            if (st != -1) { bestStart = st; bestLen = mid; lo = mid + 1; }
            else hi = mid - 1;
        }
        return s.substr(bestStart, bestLen);
    }
};
```

WATCH OUT

Arithmetic mod 2^{64} (`unsigned long long` overflow) is fast but is broken by known adversarial strings — which is why this solution verifies every hash match. Without verification, a collision returns a substring that does not actually repeat. Suffix arrays are the deterministic alternative.

38 LC 381 · Insert Delete GetRandom O(1) - Duplicates allowed

HARD Pattern 10 — Map to a set of indices

PROBLEM

LC 380, but values may repeat, and `getRandom` must return each value with probability proportional to its count.

APPROACH

The vector stores every copy, so `getRandom` is still uniform over positions. The map now sends each value to the **set of indices** holding it. Remove any one index of `val`; if it is not the last slot, move the last element into it and fix that element's index set.

> **Complexity** O(1) average per operation · O(n) space

C++ SOLUTION

```cpp
class RandomizedCollection {
    vector<int> vals;
    unordered_map<int, unordered_set<int>> where;     // value -> every index holding it
public:
    bool insert(int val) {
        bool fresh = !where.count(val);
        where[val].insert(vals.size());
        vals.push_back(val);
        return fresh;
    }
    bool remove(int val) {
        auto it = where.find(val);
        if (it == where.end()) return false;
        int i = *it->second.begin();                  // any copy will do
        it->second.erase(i);
        int lastIdx = vals.size() - 1, last = vals[lastIdx];
        if (i != lastIdx) {                           // move the last element into the hole
            vals[i] = last;
            where[last].erase(lastIdx);
            where[last].insert(i);
        }
        vals.pop_back();
        if (where[val].empty()) where.erase(val);
        return true;
    }
    int getRandom() { return vals[rand() % vals.size()]; }
};
```

> **WATCH OUT**
>
> Remove `i` from `val`'s index set **before** moving the last element: when the last element is another copy of `val`, its index set must end up with `i` and without the old last index. Erase the map entry once a value's set is empty, or `insert` will wrongly report it as already present.